xstockstrat — Cross Stock Strategies Platform

A 3–5 minute product walkthrough: what the platform does, how the services compose, the three operator UIs, and the safety properties that make it deployable to production.

Video Outline (the spine)

Time	Beat	What to show
0:00 – 0:20	**What it is.** Open-source stock strategy platform. Real-time data, indicators, backtesting, paper or live order execution.	Title card. Three-screen montage of the trader UI, insights dashboard, config UI.
0:20 – 0:50	**The pillars.** Five product areas: Trading, Market Data, Indicators, Analysis, Notifications. Each is its own service. Contracts are proto-defined.	Service map diagram (15 services, ports, language tags).
0:50 – 1:30	**Live trading core.** Order placement, fill detection, position tracking, P&L. Paper trading by default; production is gated by branch (`main` = live, `main-dev` = paper).	Trader UI placing an order, fill arriving, P&L updating.
1:30 – 2:10	**Indicators and signals.** Custom Python formula sandbox. Signal source registry. Indicator engine consumes signals from newsletters or alerting webhooks.	Indicator builder UI showing a formula, then a signal triggering a position.
2:10 – 2:50	**Analysis and backtesting.** Strategy scoring with SMA crossover, signal-weighted backtests, all via the analysis service. Insights dashboard surfaces the results.	Insights dashboard with a backtest chart and strategy score.
2:50 – 3:20	**Config as a stream.** A `WatchConfig` gRPC stream pushes live config to every service. Maintenance mode flips with one toggle in the config UI — no restarts.	Config UI flipping `platform.maintenance_mode` to true, services log "halt" within seconds.
3:20 – 3:50	**MCP agent.** External AI agents (Claude Desktop, others) talk to the platform over a Model Context Protocol server. Ingest signals, trigger backtests, query alerts.	Claude Desktop calling the MCP tool to fetch portfolio P&L.
3:50 – 4:20	**Observability.** OpenTelemetry → Grafana Cloud. Distributed traces across all 15 services. Toggleable via `OTEL_ENABLED` .	Grafana dashboard with trace waterfalls.
4:20 – 4:50	**Safety properties.** Trading mode is set at the deploy spec, not config — dev cannot accidentally trade live. Append-only ledger. `buf breaking` blocks contract regressions.	Side-by-side `.do/app.yaml` vs `.do/app.dev.yaml` showing `TRADING_MODE` divergence.
4:50 – 5:00	**Outro.** Built with agentic SDD. Open source. Link to repo.	Title card + GitHub URL.

Section 1 — Platform at a Glance

xstockstrat is a multi-service platform for designing, backtesting, and executing stock trading strategies on top of broker APIs (Alpaca today; the broker layer is pluggable).

It is built as 15 services around a single `WatchConfig` stream:

- **Backend trading core** (Go): trading, portfolio, market data
- **Analytics & signal processing** (Python): indicators, ingest, analysis
- **Platform services** (Node.js): config, identity, notify, ledger
- **Operator UIs** (Next.js): trader, insights, config UI
- **Gateway** (Nginx): unified ingress for the three UIs
- **External agent surface** (Python MCP): tools for AI agents to ingest signals, query positions, and trigger backtests

Contracts between every service are proto-defined in `packages/proto/` and code-generated to Go, Python, and TypeScript stubs. There is no service that talks to another service via untyped JSON.

Section 2 — The Operator UIs

Trader UI (port 3000)

The order entry, position monitor, and live alert surface. Authenticated via JWT through `xstockstrat-identity`. Edge-runtime middleware enforces auth; `x-user-id`, `x-access-scope`, and `x-trace-id` headers propagate downstream to every backend call.

Key features: - Symbol search → quote → order entry (market, limit, stop) - Active positions with realized + unrealized P&L - Streaming alert feed (Connect-RPC streaming, polling fallback)

Insights Dashboard (port 3001)

Strategy performance, backtest results, and indicator visualizations.

- Backtest a strategy across a historical window
- Indicator overlays on OHLCV charts
- Per-strategy scoring metrics (Sharpe, max drawdown, win rate)

Config UI (port 3002)

Runtime configuration management. Every service subscribes to `xstockstrat-config` at startup and receives a live stream of its config values. Edits in the UI propagate within one stream cycle — no restarts.

- View any key by namespace (`<service>.<category>.<key>`)
- Edit + audit trail (every change writes a `config.changed` event to the ledger)
- Emergency switch: `platform.maintenance_mode=true` halts all trading platform-wide

Section 3 — The Trading Core

Order lifecycle (`xstockstrat-trading` , Go, gRPC 50051)

```
PlaceOrder
  → risk check (xstockstrat-portfolio)
  → broker submit (Alpaca)
  → fill detection (poll-based, configurable interval)
  → ledger event (order.created, order.filled, order.rejected)
  → portfolio update (atomic position + cash adjustment)
  → notify (alert emitted to xstockstrat-notify subscribers)
```

The trading service maintains a **dual store**: in-memory order map for low-latency reads, plus TimescaleDB for durability. Fill detection runs as a background poller against Alpaca — this lets the service survive broker websocket flaps without losing fills.

Position tracking (`xstockstrat-portfolio` , Go, gRPC 50052)

- Realized P&L computed on fill: `qty × (fill_price - avg_entry_price)` for closes
- Unrealized P&L computed on read: `position × (current_price - avg_entry_price)`
- Risk check is **non-blocking** — order placement does not stall on portfolio if the latter is slow. The risk decision arrives as a separate event.

Market data (`xstockstrat-marketdata` , Go, gRPC 50053)

- Alpaca feed subscription for quotes and bars
- OHLCV storage in TimescaleDB hypertables
- Historical backfill via `TriggerBackfill` RPC for arbitrary date windows
- Indicators and analysis services pull bars from here, not directly from the broker

---

Section 4 — Indicators and Signals

Indicator engine (`xstockstrat-indicators` , Python, gRPC 50054)

A sandboxed Python formula execution environment. Users register custom formulas (RSI, MACD, custom moving averages, or arbitrary numpy/pandas expressions) and the engine evaluates them on demand against OHLCV bars from market data.

Constraints: - Sandbox limits CPU and memory per formula - Allowlisted modules: `numpy` , `pandas` , `math` , a curated indicator helper library - Formula source is stored in the indicator registry; updates require a new version

Signal source registry (Phase 3, feature 008)

A pluggable registry for external signal sources — newsletter alerts, third-party APIs, custom webhooks. Each registered source has: - A normalization adapter that maps the source's payload to the platform's signal schema - A polling interval or webhook endpoint - A health and rate-limit policy

Signal-aware indicators

Indicators can query the ingest service for active signals via `QuerySignals` . This means a "buy when RSI < 30 AND newsletter X has a bullish flag" formula is expressible — the indicator engine reaches into the signal store as part of its evaluation.

Section 5 — Analysis and Backtesting

`xstockstrat-analysis` (Python, gRPC 50056) is the strategy scoring and backtesting engine.

Built-in strategies: - **SMA crossover** — short MA crosses long MA → entry/exit signal - **Signal-weighted** — combines indicator output with active signals from the signal registry - **Custom** — user-defined strategy registered through the indicator builder

Backtesting: - Replay OHLCV bars from market data over an arbitrary historical window - Apply the strategy to generate hypothetical orders - Score the result against metrics: Sharpe ratio, max drawdown, win rate, total return

The Insights UI surfaces these scores. Backtests can also be triggered programmatically via the agent MCP server (see Section 7).

Section 6 — Config as a Stream

Every service subscribes to `xstockstrat-config` at startup via the `WatchConfig` RPC. Services block on the initial config snapshot before accepting traffic. The stream stays open for the lifetime of the service — config changes propagate within one stream cycle.

Key naming convention: `<service>.<category>.<key>`. Examples: - `trading.risk.max_position_size` — float, default 1000 - `indicators.formula.timeout_ms` — int, default 5000 - `platform.maintenance_mode` — bool, default false

Sensitive values use the `secret.*` prefix and are never logged or shipped to telemetry.

Production safety: trading mode (`TRADING_MODE`, `ALPACA_PAPER`, `ALPACA_BASE_URL`) is **not** in the config stream. It is hard-set at the deploy-spec level (`.do/app.yaml` vs `.do/app.dev.yaml`). This means: - Dev (`main-dev` branch) → paper trading, always - Prod (`main` branch) → live trading, always - No config UI edit can flip a dev environment into live trading

---

Section 7 — The MCP Agent

`xstockstrat-agent` (port 9000, SSE transport) is a Model Context Protocol server. External AI agents — Claude Desktop, custom Claude API integrations, anything that speaks MCP — connect to it to interact with the platform.

Exposed tools: - **Ingest signals** — submit a newsletter or alert payload, the agent normalizes and forwards to `xstockstrat-ingest` - **Trigger backtests** — kick off an `xstockstrat-analysis` backtest with a strategy + date window - **Query positions** — current portfolio holdings and P&L - **Query alerts** — recent notifications from the notify service - **Place paper orders** — convenience wrapper for the trading service

The agent forwards an `x-mcp-secret` header on outbound calls (`MCP_AGENT_SECRET` env var) to identify itself. Platform services trust this header as "request originated from the AI agent surface."

Section 8 — Observability

OpenTelemetry SDK in every service exports OTLP traces, metrics, and logs to Grafana Cloud (or any OTLP receiver). Toggle: `OTEL_ENABLED=true`.

- Distributed traces span all 15 services via `x-trace-id` header propagation
- Per-language instrumentation: `internal/telemetry/` (Go), `app/telemetry.py` (Python), `src/telemetry.ts` (Node.js)
- OTel init errors **never** prevent service startup — observability is best-effort

Configured dashboards: trade flow (order → fill → ledger event), config stream health, indicator latency percentiles, alert delivery latency.

Section 9 — Storage and Event Sourcing

Event ledger (`xstockstrat-ledger` , Node.js, gRPC 50057)

An append-only event store. Every state-changing action in the platform writes an event: - `order.created` , `order.filled` , `order.rejected` - `position.opened` , `position.closed` - `config.changed` - `alert.emitted` - `signal.ingested`

Events are immutable. Replay is possible from any point. The ledger is the source of truth for "what happened."

Per-service schemas in TimescaleDB

Each service owns a Postgres schema in shared TimescaleDB: - `trading.*` , `portfolio.*` , `marketdata.*` , `indicators.*` , `ingest.*` , `analysis.*` , `ledger.*` , `identity.*` , `config.*` - Migrations live in `services/<service>/migrations/` and run via golang-migrate - Hypertables for time-series data (OHLCV bars, signal events, indicator results)

Section 10 — Authentication and Authorization

`xstockstrat-identity` (Node.js, gRPC 50058) handles JWT issuance, API key management, and auth verification.

- JWT tokens are issued at login (`/api/auth/login` on any Next.js UI)
- Tokens include minimal claims: `sub` (user id), `scope` (access scope), `exp`
- API keys for programmatic access are scoped (read-only, trading, admin) and rotatable
- Every backend service uses the same JWT verification middleware via the identity gRPC stub
- The `x-access-scope` header propagates the caller's scope through every downstream call

Nginx strips client-supplied `x-user-id` , `x-access-scope` , and `x-trace-id` headers on inbound requests. Inside the platform, those headers are trusted because only the platform itself can have set them.

Section 11 — Safety Properties

What makes this deployable to production with real money:

1. **Trading mode is platform-level, not config-level.** No runtime knob can flip a dev environment into live trading.

2. **Append-only ledger.** Every state change is recorded immutably. Auditable history of every order, position, and config change.
3. **Risk check before broker submit.** Position sizing is checked against configured limits before any order leaves the platform.
4. **Maintenance mode.** A single config flag halts all trading platform-wide. Takes effect within one `WatchConfig` stream cycle. No restart required.
5. **Proto contract gates.** `buf breaking` blocks contract regressions in CI. Stub freshness check blocks committed-stub drift.
6. **Branch-protected deploys.** `main-dev` → dev (paper). `main` → prod (live). Both branches require PR review and CI green before merge.
7. **Secret hygiene.** No secrets in config. `secret.*` namespace for sensitive values. Public-repo secret scanning (trufflehog + gitleaks) runs on every PR.

Section 12 — Built With Agentic SDD

Every feature in this repo — from the proto contracts to the Next.js dashboards to the CI workflows — was built through an agentic Spec-Driven Development loop. Open `docs/roadmap/features/` and browse any feature's `context.md` to see the session-by-session record of agents writing code under human gates.

This is not a curated public release of a privately-built codebase. It is the actual codebase, with the actual agent-collaborative history.

Outro

Repository: `github.com/davcs86/xstockstrat` **Architecture reference:** `CLAUDE.md` (root) and `docs/CLAUDE.md` **Quick start:** `docs/setup/getting-started.md` **Built with:** Claude Code · Anthropic Claude API · OpenTelemetry · TimescaleDB · Alpaca Markets · DigitalOcean App Platform